

CSCE 421: Machine Learning

Homework #3

March 20, 2025

Sutton Elliott

UIN:531008819

Email: suelliott77@tamu.edu

Describe what are hinge loss, logistic regression loss, and 0-1 loss mathematically. Describe their similarities and differences using the unified picture we developed in class.

Hinge Loss:

Hinge Loss is commonly used in Support Vector Machines (SVMs) for classification. It is defined as:

$$L_{hinge}(y, f(x)) = \max(0, 1 - yf(x))$$

where:

- $y \in \{-1, 1\}$ is the true label,
- $f(x)$ is the predicted score (a real-values function).

This loss encourages correct classification with a margin of at least 1.

Logistic Regression Loss:

Logistic regression uses the negative log-likelihood loss, also called log loss or cross- entropy loss, defined as:

$$L_{logistic}(y, f(x)) = \log(1 + e^{-yf(x)})$$

where:

- $y \in \{-1, 1\}$ is the true label,
- $f(x)$ is the predicted score (typically the logit, $f(x) = wTx$).

This loss is used in probabilistic models and is differentiable everywhere.

0-1 Loss:

The 0-1 loss is the most natural loss function for classification but is not convex or differentiable. It is defined as:

$$L_{0-1}(y, f(x)) = 1(yf(x) \leq 0)$$

Where $1(*)$ is the indicator function, which is 1 if the condition is true and 0 otherwise.

Similarities and Differences:

We can view all three loss functions under a common framework in terms of how they penalize misclassification:

- All three measure misclassification in terms of $y f(x)$, the signed margin.
- 0-1 loss only cares about whether the classification is correct (discrete loss).
- Hinge loss gives a piecewise linear penalty for incorrect classification and enforces a margin.
- Logistic loss gives a smooth exponential-like penalty and has probabilistic interpretation.

Comparison of Loss Curves:

- Hinge loss (SVM) enforces a margin, penalizing points within the margin but ignoring correct classifications beyond it.
- Logistic loss (Logistic Regression) penalizes misclassifications more smoothly and is always differentiable.
- 0-1 loss is ideal but impractical for gradient-based optimization since it is not continuous.

In conclusion, Hinge loss and logistic loss are convex and used in gradient-based optimization, while 0-1 loss is not. Hinge loss enforces a margin, while logistic loss allows probabilistic interpretation. Logistic loss and hinge loss approximate 0-1 loss, but hinge loss encourages a margin-based separation, whereas logistic loss models probabilities.

By relying on the result in the above question, consider a data point that is correctly classified and distant from the decision boundary. Why would SVM's decision boundary be unaffected by this point, but the one learned by logistic regression be affected?

A Support Vector Machine (SVM) and Logistic Regression both aim to find a decision boundary for classification, but they differ in how they handle correctly classified points that are far from the boundary.

SVM:

- SVM seeks to maximize the margin between classes.
- It relies only on support vectors—the points closest to the decision boundary.
- Once a point is correctly classified and lies beyond the margin, it has no effect on the boundary.
- As a result, a distant correctly classified point does not impact the SVM's decision boundary.

Logistic Regression:

- Logistic regression finds a decision boundary by maximizing the likelihood of the observed data.
- It considers all data points, not just those near the boundary.
- Since each point contributes to the likelihood function, even correctly classified points far from the boundary can influence the model's parameters.
- This means that logistic regression's decision boundary can shift based on these distant points.

SVM's decision boundary is only influenced by support vectors, while logistic regression's boundary is influenced by all training points even those far from the decision boundary.

Exercise 7.8 (e-Chap:7-15) in the book “Learning from Data”. For your convenience, the question is repeated below:

Exercise 7.8: Repeat the computations in Example 7.1 for the case where the output transformation is the identity. You should compute $s(\ell)$, $x(\ell)$, $\delta(\ell)$ and $\partial e / \partial W(\ell)$. Note the “output transformation” refers to the nonlinear function in the output layer only.

The weight matrices are the same as in example 7.1. For the data point $x=2, y=1$, we have

$$x^{(0)} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

, $s^{(1)}$ is the same as before,

$$s^{(1)} = W^{(1)}x^{(0)} = \begin{bmatrix} 0.7 \\ 1 \end{bmatrix}$$

So $x^{(1)} = \theta(s^{(1)}) = \begin{bmatrix} 1 \\ 0.7 \\ 2 \end{bmatrix}$ (this will include the bias term), and $s^{(2)} = [-2.1]$,

$$x^{(2)} = \begin{bmatrix} 1 \\ -2.1 \end{bmatrix}$$

$$s^{(3)} = [-3.2],$$

$$x^{(3)} = -3.2$$

Apply backpropagation and note that $\theta'(s^{(l)}) = 1$ to compute

$$\delta^{(3)} = 2(x^{(3)} - y) = -8.4$$

$$\delta^{(3)} = \theta'(s^{(2)}) \otimes [W^{(3)}\delta^{(3)}] = 16.8$$

$$\delta^{(1)} = \begin{bmatrix} -16.8 \\ 50.4 \end{bmatrix}$$

The formula used in each of the following calculations is as follows: $\frac{\partial e}{\partial W^{(l)}} = x^{(l-1)}(\delta^{(l)})^T$

$$\frac{\partial e}{\partial W^{(1)}} = x^{(0)}(\delta^{(1)})^T = \begin{bmatrix} -16.8 & 50.4 \\ -33.6 & 100.8 \end{bmatrix}$$

$$\frac{\partial e}{\partial W^{(2)}} = x^{(1)}(\delta^{(2)})^T = \begin{bmatrix} 16.8 \\ 11.76 \\ 33.6 \end{bmatrix}$$

$$\frac{\partial e}{\partial W^{(3)}} = x^{(2)}(\delta^{(3)})^T = \begin{bmatrix} -8.4 \\ 17.64 \end{bmatrix}$$

You'd like to train a fully connected neural network with 5 hidden layers, each with 10 hidden units (i.e., $d(l) = 10$ for these hidden layers using the notations in the textbook). The input is 20-dimensional, and the output is a scalar. What is the total number of trainable parameters in your network (include bias in your calculation)?

Input Layer: $d(0) = 20$ (20-dimensional input)

Hidden Layers: 5 hidden layers, each with $d(l) = 10$ units

Output Layer: Scalar Output, $d(6) = 1$

Each layer l has:

A weight matrix of size $d(l) \times d(l-1)$

A bias vector of size $d(l) \times 1$

For each layer:

Layer 1 –

- Weights: $10 \times 20 = 200$
- Biases: 10
- Total: $200 + 10 = 210$

Layer 2 –

- Weights: $10 \times 10 = 100$
- Biases: 10
- Total: $100 + 10 = 110$

Layer 3 –

- Weights: $10 \times 10 = 100$
- Biases: 10
- Total: $100 + 10 = 110$

Layer 4 –

- Weights: $10 \times 10 = 100$
- Biases: 10
- Total: $100 + 10 = 110$

Layer 5 –

- Weights: $10 \times 10 = 100$
- Biases: 10
- Total: $100 + 10 = 110$

Output Layer

- Weights: $1 \times 10 = 10$
- Biases: 1
- Total: $10 + 1 = 11$

Final Total:

$$210 + 110 + 110 + 110 + 110 + 11 = 651$$

There is total of 651 trainable parameters.

Multi-layer perceptron models for Handwritten Digits Classification: In this assignment, you will implement the multi-layer perceptron model using PyTorch on a partial dataset from MNIST. In this classification task, the model will take a 16×16 image of handwritten digits as inputs and classify the image into three different classes (0,1,2). The “data” folder contains the dataset which has already been split into a training set and a testing set. All data examples are saved in dictionary-like objects using “npz” file. For each data sample, the dictionary key ‘x’ indicates its raw features, which are represented by a 256- dimensional vector where the values between $[-1, 1]$ indicate grayscale pixel values for a 16×16 image. In addition, the key ‘y’ is the label for a data example, which can be 0, 1, or 2. The “code” folder provides the starting code (main training logic has already been implemented). You must implement the models using the starting code. Please learn the PyTorch tutorial before starting this assignment: <https://pytorch.org/tutorials/beginner/basics/intro.html>

In the function prepare X from “code/DataReader.py”, please copy your implementation from HW2, and don’t change other parts in “code/DataReader.py”. Then, implement the multi-layer perceptron model in the “code/MLP.py”. Please use only one hidden layer in the MLP (Hint: you can use torch.nn.Linear class in PyTorch). In this assignment, we will use a new nonlinear function ReLU that will be discussed later in this class (PyTorch has implemented this function, so you do not need to be concerned about how to implement it). It outputs the same value if the input is positive and outputs zero if the input is negative (Hint: use torch.nn.functional.relu). The sequential layers are:

Linear $\rightarrow$ ReLU $\rightarrow$ Linear (3 output channels)

Function prepare X copied from HW2

MLP.py implemented

Implement evaluation function in “code/main.py”. This function should evaluate classification accuracy of the trained model on the test set. Please remember to load the best model checkpoint (based on validation accuracy) before running the evaluation on the test set.

Evaluation function implemented

Tune the HIDDEN SIZE hyperparameter in “code/main.py” to find the best model performance on the test set. Please include the HIDDEN SIZE you’ve tried and the corresponding classification accuracy on the test set in your submission.

Below are the Hidden Sizes and corresponding Test Accuracy for each one:

Hidden Size	Test Accuracy
3	0.6906
16	0.8672
32	0.8721
64	0.8672

128	0.8660
256	0.8587
512	0.8563
1024	0.8563
2048	0.8563
20	0.8709
24	0.8709
28	0.8697
8	0.8660
12	0.8685
768	0.8563
912	0.8563

Conclusions drawn from the values chosen:

The values in red are the initial values I chose to test a wide range. This took care of all kinds of capacity from small to medium to large. The highest test accuracy being from a hidden size of 32 with 0.8721. Another conclusion is that as the hidden size increased, the test accuracy seemed to flatten out around 0.8563, getting that value for 3 different values from 512, 1024, and 2048.

The values in blue were chosen to further prove the conclusions that we drew from the initial values in red. I chose a couple other values above 512, which again yielded a result of 0.8563, which seems to be where things flatten out above approximately 512. The other conclusion that is further proven is that the most optimal value for hidden size is between 16 and 32 with the highest test accuracy still coming from a hidden size of 32 with a test accuracy of 0.8721.

Based on the results, 32 is the best choice for hidden size. It provides the highest test accuracy and avoids the overfitting or plateauing seen with the larger hidden sizes. Below is my terminal with the test accuracy and hidden size of 32.

```

21 # Hyperparameters
22 EPOCHS = 50
23 LEARNING_RATE = 0.001
24 HIDDEN_SIZE = 32
25 BATCH_SIZE = 32
26
27 def evaluation(model, best_model_state, best_val_acc):
28     ### YOUR CODE HERE

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

Epoch [37/50], Loss: 0.2962, Val Accuracy: 0.8698
Epoch [38/50], Loss: 0.2948, Val Accuracy: 0.8698
Epoch [39/50], Loss: 0.2935, Val Accuracy: 0.8698
Epoch [40/50], Loss: 0.2923, Val Accuracy: 0.8698
Epoch [41/50], Loss: 0.2912, Val Accuracy: 0.8698
Epoch [42/50], Loss: 0.2902, Val Accuracy: 0.8698
Epoch [43/50], Loss: 0.2893, Val Accuracy: 0.8698
Epoch [44/50], Loss: 0.2885, Val Accuracy: 0.8698
Epoch [45/50], Loss: 0.2877, Val Accuracy: 0.8698
Epoch [46/50], Loss: 0.2870, Val Accuracy: 0.8698
Epoch [47/50], Loss: 0.2864, Val Accuracy: 0.8698
Epoch [48/50], Loss: 0.2858, Val Accuracy: 0.8698
Epoch [49/50], Loss: 0.2852, Val Accuracy: 0.8698
Epoch [50/50], Loss: 0.2846, Val Accuracy: 0.8698
Test Accuracy: 0.8721
Training complete. Best model saved as best_mlp_model.pth.
PS C:\Users\sutto\OneDrive\Desktop\CSCE 421\HW3>

```